
ARCHITECTURE BLUEPRINT · THE BANNER SUBSYSTEM

Necoyoad

v9

*Banners, banner items, jQuery slider plugins,
and the 33-template rendering system*

`scope:` banner/banner_item tables, admin controller/model,
storefront banner widget, 33 slider templates,
ntPlugins registry, per-item widget composition
`findings:` jquery_plugin discriminator, ntPlugins JS registry,
per-item widget overlays, params column unused
`listings:` 8 PHP · `pages:` 13

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — The Banner Subsystem (v9)

*Banners, banner items, jQuery slider plugins,
and the 33-template rendering system*

Abstract

This is the ninth volume of the Necoyoad architecture blueprint. v9 covers the banner subsystem: how the admin creates banners with a `jquery_plugin` discriminator, how banner items (slides) carry images, links, and localised descriptions, how the storefront banner widget module loads the banner and dynamically selects the matching slider template, how the `ntPlugins` JavaScript registry initialises the jQuery slider, and how per-item widgets can be attached to individual slides.

The banner subsystem is the most visually diverse part of Necoyoad: 33 slider/gallery templates in the `choroni/banner/` folder, each backed by a jQuery plugin (NivoSlider, Slick, Camera, EisSlideshow, Fancybox, etc.) with its own JS and CSS assets. The `jquery_plugin` column on the `banner` table is the discriminator: the storefront controller reads it, loads the matching `.tpl` file, enqueues the matching `slider.js` and `slider.css`, and the template pushes a configuration object into the `window.ntPlugins` registry, which `theme.js::loadNTPlugins()` initialises on DOM ready.

v9 documents all of this with 8 PHP code listings, 3 TikZ diagrams, the full 33-template inventory, and a banner rendering lifecycle trace. The stance is unchanged: **descriptive, not prescriptive**.

Keywords: Necoyoad, OpenCart, PHP 8.1, banner, banner item, slider, jQuery plugin, NivoSlider, Slick, Camera, `ntPlugins`, `loadNTPlugins`, widget composition, polymorphic description, EAV property.

Contents

1	Introduction	3
1.1	Nine Volumes	3
1.2	Scope	3
2	The Banner Schema	3
2.1	The <code>banner</code> Table	3
2.2	The <code>banner_item</code> Table	4
3	The Admin Banner Controller and Model	4
3.1	The Admin Controller	4
3.2	The Admin Model	4
4	The Storefront Banner Widget Module	5
4.1	The <code>module:settings</code> Filter	5
4.2	The Per-Item Widget Composition	7
4.3	The <code>carousel()</code> AJAX Endpoint	7
5	The Storefront Banner Model	7
6	The 33 Slider/Gallery Templates	8
6.1	Template Structure	9
6.2	The <code>ntPlugins</code> Registry and <code>loadNTPlugins()</code>	10
7	The Banner Rendering Lifecycle	10
7.1	Admin Creates a Banner	10
7.2	Storefront Renders the Banner Widget	11
8	Cross-Cutting Findings	11
8.1	The Banner Subsystem is a Plugin-Aware Widget	11
8.2	The <code>ntPlugins</code> Registry is the JS Initialisation Seam	12
8.3	Per-Item Widget Composition is Unique to Banners	12
8.4	The <code>params</code> Column is Effectively Unused	12
8.5	The <code>banner_items</code> Form Array Carries Everything	12
9	Closing Observations	12
9.1	The Banner Subsystem is the Most Visually Diverse Part	12
9.2	The Per-Item Widget Composition is a Unique Feature	12
9.3	Final Note	12
A	Glossary	13

1 Introduction

1.1 Nine Volumes

v1–v8 covered the whole platform, source verification, the rendering pipeline, the marketing subsystem, multi-store/multi-language, the admin back-office, the menu links, and the CMS posts/pages. v9 covers the banner subsystem.

1.2 Scope

v9 covers:

- The `banner` and `banner_item` tables.
- The admin banner controller (`ControllerContentBanner`) and model (`ModelContentBanner`).
- The storefront banner widget module (`ControllerModuleBanner`).
- The storefront banner model (with store-scoped query and publish-window check).
- The `jquery_plugin` discriminator and dynamic template selection.
- The 33 slider/gallery templates in `choroni/banner/`.
- The `ntPlugins` JavaScript registry and `loadNTPlugins()` initialiser.
- The per-item widget composition (widgets attached to individual banner items).
- The `carousel()` AJAX endpoint.

2 The Banner Schema

2.1 The banner Table

```
1 CREATE TABLE 'banner' (  
2   'banner_id'          int(11) NOT NULL,  
3   'name'               varchar(250) NOT NULL,  
4   'jquery_plugin'      varchar(150) NOT NULL, -- e.g. 'nivo-slider',  
                        'slick', 'camera-v1.3.4'  
5   'params'             text NOT NULL,          -- serialised PHP slider  
                        config  
6   'publish_date_start' date NOT NULL,  
7   'publish_date_end'   date NOT NULL,  
8   'status'             int(1) NOT NULL,  
9   'date_added'         datetime NOT NULL,  
10  'date_modified'      datetime NOT NULL  
11 );
```

Listing 1: The `banner` table schema.

The `jquery_plugin` column is the key discriminator: it names the jQuery slider plugin to use for rendering. The storefront controller reads this column and uses it to select the matching template file (`choroni/banner/<jquery_plugin>.tpl`), the matching JS file (`web/assets/js/sliders/<jquery_plugin>/slider.js`), and the matching CSS file (`web/assets/css/sliders/<jquery_plugin>/slider.css`).

The `params` column stores a serialised PHP array of slider configuration (effect, speed, transition type, etc.). However, the storefront templates hardcode the configuration in the `ntPlugins.push()` call rather than reading from `params` — so `params` is effectively unused by the storefront (it may be used by the admin form for persistence).

2.2 The banner_item Table

```
1 CREATE TABLE 'banner_item' (  
2   'banner_item_id' int(11) NOT NULL,  
3   'banner_id'      int(11) NOT NULL,  
4   'image'          varchar(250) NOT NULL, -- filesystem path to the slide  
5   image  
6   'link'           varchar(250) NOT NULL, -- URL the slide links to  
7   'sort_order'     int(11) NOT NULL,  
8   'status'         int(1) NOT NULL DEFAULT '1'  
9 );
```

Listing 2: The banner_item table schema.

Each banner item is a single slide: an image, a link, a sort order, and a status. The slide's localised title and description are stored in the polymorphic `description` table with `object_type` = 'banner_item' and `object_id` = <banner_item_id>. Per-item properties (like `offsetX`, `offsetY` for positioning overlaid widgets) are stored in the EAV `property` table with the same (`object_id`, `object_type`) pair.

3 The Admin Banner Controller and Model

3.1 The Admin Controller

`app/admin/controller/content/banner.php` (283 lines) extends `ControllerAdmin` (v6) with standard declarative CRUD. The `$form_vars` array declares the expected POST fields:

```
1 protected array $form_vars = [  
2   'banner_id'      => ['name' => 'banner_id', 'type' => 'number'],  
3   'name'          => ['name' => 'name', 'type' => 'string'],  
4   'jquery_plugin' => ['name' => 'jquery_plugin', 'type' => 'string'],  
5   'publish_date_start' => ['name' => 'publish_date_start', 'type' =>  
6     'date'],  
7   'publish_date_end' => ['name' => 'publish_date_end', 'type' => 'date'],  
8   'banner_stores'  => ['name' => 'banner_stores', 'type' => 'array'],  
9   'banner_items'   => ['name' => 'banner_items', 'type' => 'array'],  
10  'banner_properties' => ['name' => 'banner_properties', 'type' =>  
11    'array'],  
12  'stores'          => ['name' => 'stores', 'type' => 'array'],  
13 ];
```

Listing 3: `ControllerContentBanner` — declarative form vars (abridged).

The `banner_items` array carries the slide data (image, link, sort_order, status, descriptions, properties) from the admin form. The `banner_stores` array carries the store assignments (for `object_to_store`). Because the controller extends `AdminController`, all CRUD is inherited.

3.2 The Admin Model

`app/admin/model/content/banner.php` (324 lines) declares `$table` = "banner", `$object_type` = "banner", and `$description_object_type` = "banner_item". The `on('save')` hook manages items:

```
1 public function init() {  
2   $this->on("save", function ($args) {  
3     $data = $args[0];  
4     if ($data['items']) {  
5       foreach ($data['items'] as $key => $item) {  
6         $item['banner_id'] = $data['id'];  
7         $item['sort_order'] = $key;
```

```

8         $this->setItem($item);
9     }
10 }
11 });
12
13 $this->on("delete", function ($args) {
14     $data = $args[0];
15     $this->deleteItems($data['id']);
16 });
17 }

```

Listing 4: ModelContentBanner — the on('save') hook for items.

The `setItem()` method handles both insert and update of a banner item: it checks if `banner_item_id` exists, updates or inserts accordingly, then saves the descriptions (localised title and HTML description) and properties (EAV) for the item.

The `deleteItems()` method cascades: deletes description rows, property rows, and `banner_item` rows for the given `banner_id`.

The `getById()` method loads a banner with its items, stores, and properties:

```

1 public function getById($id) {
2     $results = parent::getAll(array('banner_id'=>$id));
3     $results[0]['banner_items'] = $this->getItems(array('banner_id'=>$id));
4     $results[0]['banner_stores'] = $this->getStores($id);
5     $results[0]['banner_properties'] = $this->getAllProperties($id);
6     return $results[0];
7 }

```

Listing 5: ModelContentBanner::getById() — loads banner with items.

4 The Storefront Banner Widget Module

`app/shop/controller/module/banner.php` (113 lines) is the storefront widget module. It extends `ControllerModuleModuleController` (v3) and registers a `module:settings` filter in `init()`.

4.1 The module:settings Filter

The filter is the heart of the banner widget. It:

1. Loads the banner by `settings['banner_id']` via `modelBanner->getById()`.
2. Loads the banner items via `modelBanner->getItems()`.
3. For each item, loads its localised descriptions and its per-item widgets (via a recursive `NecoWidget::getWidgets()` call with `object_type = 'banner_item'` and `object_id = <banner_item_id>`).
4. Enqueues the slider's JS file (`web/assets/js/sliders/<plugin>/slider.js`).
5. Enqueues the slider's CSS file (`web/assets/css/sliders/<plugin>/slider.css`).
6. Selects the template: `choroni/banner/<jquery_plugin>.tpl` (or the active theme's version, or falls back to `nivo-slider.tpl`).

```

1 public function init() {
2     $this->addFilter("module:settings", function ($data) {
3         $settings = $data['settings'];
4         if (isset($settings['banner_id'])) {

```

```

5      $this->load->model('content/banner');
6      $this->data['banner'] =
          $this->modelBanner->getById((int)$settings['banner_id']);
7
8      if (isset($this->data['banner']['banner_id'])) {
9          $items =
10             $this->modelBanner->getItems($this->data['banner']['banner_id']);
11         foreach ($items as $k => $item) {
12             // Load localised descriptions
13             $items[$k]['descriptions'] =
14                 $this->modelBanner->getDescriptions($item['banner_item_id']);
15
16             // Load per-item widgets (widgets attached to this slide)
17             $w = new NecoWidget($this->registry, $this->Route);
18             $params = array(
19                 'store_id' => STORE_ID,
20                 'landing_page' => 'all',
21                 'object_type' => 'banner_item',
22                 'object_id' => $item['banner_item_id'],
23                 'position' => 'main'
24             );
25             $widgets = $w->getWidgets($params, !$this->user->getId());
26             foreach ($widgets as $v) {
27                 $s = (array)unserialize($v['settings']);
28                 $this->children[$v['name']] = $s['route'];
29                 $this->widget[$v['name']] = $v;
30                 $items[$k]['widgets'][$v['name']] = array(
31                     'name' => $v['name'],
32                     'offsetY' => $s['offsetY'],
33                     'offsetX' => $s['offsetX']
34                 );
35             }
36         }
37         $this->data['banner']['items'] = $items;
38     }
39
40     // Select template based on jquery_plugin
41     if (!empty($this->data['banner']['jquery_plugin'])) {
42         $plugin = $this->data['banner']['jquery_plugin'];
43         // Enqueue JS
44         if (file_exists(DIR_JS . 'sliders/' . $plugin .
45             '/slider.js')) {
46             $this->javascripts[$plugin] = HTTP_JS . 'sliders/' .
47                 $plugin . '/slider.js';
48         }
49         // Enqueue CSS
50         if (file_exists(DIR_CSS . 'sliders/' . $plugin .
51             '/slider.css')) {
52             $this->styles[] = ['href' => HTTP_CSS . 'sliders/' .
53                 $plugin . '/slider.css', 'media' => 'all'];
54         }
55         // Select template
56         if (file_exists(DIR_TEMPLATE .
57             $this->config->get('config_template') . '/banner/' .
58             $plugin . '.tpl')) {
59             $this->template = $this->config->get('config_template') .
60                 '/banner/' . $plugin . '.tpl';
61         } elseif (file_exists(DIR_TEMPLATE . 'choroni/banner/' .
62             $plugin . '.tpl')) {
63             $this->template = 'choroni/banner/' . $plugin . '.tpl';
64         } else {
65             $this->template = 'choroni/banner/nivo-slider.tpl';
66         }
67     }

```

```

58     }
59     return $data;
60 });
61 }

```

Listing 6: ControllerModuleBanner::init() — the module:settings filter (abridged).

4.2 The Per-Item Widget Composition

Banners support per-item widget composition

Each banner item (slide) can have its own set of widgets, loaded via a recursive `NecoWidget::getWidgets()` call with `object_type = 'banner_item'` and `object_id = <banner_item_id>`. These widgets are registered as children of the banner widget controller (merged into `$this->children`), and their `offsetX/offsetY` settings are stored in the `$items[$k]['widgets']` array for the template to position them as overlays on the slide. This means a single banner slide can have overlaid widgets (e.g. a “product overview” widget positioned at `offsetX=100, offsetY=50` over the slide image) — the same widget composition mechanism that drives page-level composition (v3, v8) is applied at the slide level.

4.3 The carousel() AJAX Endpoint

The banner module also provides a `carousel()` method that returns banner items as JSON (with thumbnail resizing via `NTImage::resizeAndSave()`). This is used by JavaScript carousels that load their slides via AJAX rather than server-side rendering:

```

1 public function carousel() {
2     $this->load->model('content/banner');
3     $this->load->auto('image');
4     $this->load->auto('json');
5
6     $banner =
7         $this->modelBanner->getById($this->request->getQuery('banner_id'));
8     $json['results'] = $banner['items'];
9
10    $width = isset($_GET['width']) ? $_GET['width'] : 80;
11    $height = isset($_GET['height']) ? $_GET['height'] : 80;
12    foreach ($json['results'] as $k => $v) {
13        $json['results'][$k]['thumb'] = NTImage::resizeAndSave($v['image'],
14            $width, $height);
15        $json['results'][$k]['title'] = $v['title'];
16        $json['results'][$k]['link'] = $v['link'];
17    }
18
19    $this->response->setOutput(Json::encode($json),
20        $this->config->get('config_compression'));
21 }

```

Listing 7: ControllerModuleBanner::carousel() — the AJAX endpoint.

5 The Storefront Banner Model

`app/shop/model/content/banner.php` (63 lines) provides storefront read access to banners with store-scoping and publish-window checks:

```

1 public function getById($banner_id) {

```



```

2     $query = $this->db->query("SELECT DISTINCT * FROM banner b
3         LEFT JOIN object_to_store b2s ON
4             (b.banner_id = b2s.object_id AND b2s.object_type = 'banner')
5         WHERE b.banner_id = '" . (int)$banner_id . "'
6         AND b.publish_date_start <= NOW()
7         AND (b.publish_date_end >= NOW() OR b.publish_date_end = '0000-00-00')
8         AND b.status = '1'
9         AND b2s.store_id = '" . (int)STORE_ID . "'");
10
11     $return = [];
12     if ($query->num_rows) {
13         $return = array_merge($query->row, array('items' =>
14             $this->getItems($banner_id)));
15     }
16     return $return;
17 }
18
19 public function getItems($banner_id) {
20     $sql = "SELECT * FROM banner_item bi
21         WHERE bi.banner_id = '" . (int)$banner_id . "'
22         AND bi.status = '1'";
23     return $this->db->query($sql)->rows;
24 }

```

Listing 8: Storefront ModelContentBanner::getById() — store-scoped with publish window.

The query checks:

- `publish_date_start <= NOW()` — the banner's publish window has started.
- `publish_date_end >= NOW() OR publish_date_end = '0000-00-00'` — the banner's publish window has not ended (or has no end date).
- `status = '1'` — the banner is enabled.
- `b2s.store_id = STORE_ID` — the banner is assigned to the current store (via the `object_to_store` junction, v5).

The items query only returns items with `status = '1'` (enabled slides).

6 The 33 Slider/Gallery Templates

The `choroni/banner/` folder contains 33 templates, each backed by a jQuery plugin:

Table 1: The 33 banner templates.

Template	Slider/gallery type
<code>nivo-slider.tpl</code>	NivoSlider (the default fallback).
<code>nivo-slider-v3.1.tpl</code>	NivoSlider v3.1.
<code>slick.tpl</code>	Slick carousel.
<code>camera-v1.3.4.tpl</code>	Camera (jQuery slideshow) v1.3.4.
<code>eislideshow.tpl</code>	EIS Slideshow.
<code>evolution-v1.1.5.tpl</code>	Evolution slider v1.1.5.
<code>layer-slider-v0.0.1.tpl</code>	Layer slider v0.0.1.
<code>parallax-content-slider.tpl</code>	Parallax content slider.
<code>horizontal-parallax.tpl</code>	Horizontal parallax slider.
<code>slicebox-v1.1.0.tpl</code>	Slicebox 3D slider v1.1.0.
<code>jssor-vertical-nav.tpl</code>	Jssor slider with vertical navigation.

Template	Slider/gallery type
necosl原因er.tpl	Necoyoad's own slider (custom).
carousel.tpl	Simple carousel.
horizontal.tpl	Horizontal slider.
vertical.tpl	Vertical slider.
fancybox-gallery.tpl	Fancybox lightbox gallery.
fancybox-grid.tpl	Fancybox grid gallery.
grid-gallery.tpl	CSS grid gallery.
horizontal-hover_effect-01 through -15	15 horizontal hover-effect gallery variants.

6.1 Template Structure

Each slider template follows the same pattern:

```

1  <?php $tpl = is_dir(DIR_TEMPLATE. $this->config->get('config_template'))
   ."/shared")
2      ? $this->config->get('config_template') : "choroni"; ?>
3  <li id="<?php echo $widgetName; ?>" class="banner nivo nt-editable"
4      data-banner="nivoSlider">
5
6  <?php if (count($banner['items'])) { ?>
7      <div class="content">
8          <!-- The slider container -->
9          <div id="slider" class="nivoSlider">
10             <?php foreach ($banner['items'] as $item) { ?>
11                 get('config_language_id')]['title']
14                         ?>"
15                     title="<?php echo
16                         $item['descriptions'][$Config->get('config_language_id')]['title']
17                         ?>" />
18             <?php } ?>
19         </div>
20     </div>
21
22     <!-- Push slider config to the ntPlugins registry -->
23     <script>
24     $(function(){
25         ntPlugins = window.ntPlugins || [];
26         ntPlugins.push({
27             id: "<?php echo $widgetName; ?> .nivoSlider",
28             config: {
29                 effect: 'random',
30                 slices: 12,
31                 animSpeed: 300,
32                 pauseTime: 6000,
33                 // ... more slider-specific options
34             },
35             plugin: 'nivoSlider'
36         });
37         window.ntPlugins = ntPlugins;
38
39         if (typeof loadNTPlugins !== 'undefined' && typeof loadNTPlugins ===
40             'function') {
41             loadNTPlugins();
42         }
43     });
44     </script>

```

```
40 <?php } ?>
41 </li>
```

Listing 9: The universal slider template pattern (abridged from `nivo-slider.tpl`).

The template:

1. Iterates `$banner['items']` and emits the slider's HTML structure (e.g. `<img>` tags inside a `<div class="nivoSlider">`).
2. Uses `$Config->get('config_language_id')` to select the localised title/description for each slide (from the polymorphic `description` table).
3. Uses `$Image->resizeAndSave()` for thumbnail generation.
4. Pushes a configuration object into `window.ntPlugins` with the slider's jQuery plugin name and options.
5. Calls `loadNTPlugins()` to initialise the plugin.

6.2 The ntPlugins Registry and loadNTPlugins()

The `ntPlugins` registry (v3, §2.5) is a JavaScript array that collects slider initialisation configurations. Each template pushes an entry with:

- `id` — the CSS selector for the slider container.
- `config` — the slider-specific options (effect, speed, transition, etc.).
- `plugin` — the jQuery plugin method name (e.g. `'nivoSlider'`, `'slick'`).

`theme.js::loadNTPlugins()` iterates the array and, for each entry, calls `$(item.id)[item.plugin](item.config)` — i.e. it applies the jQuery plugin to the slider container with the configuration. After processing, each entry is set to `null` to prevent re-initialisation.

This decoupled registry pattern allows multiple sliders on the same page (each with its own plugin and config) without the templates needing to know about each other.

7 The Banner Rendering Lifecycle

7.1 Admin Creates a Banner

1. Admin navigates to `?r=content/banner/insert`.
2. `ControllerContentBanner::insert()` (inherited from `AdminController`) calls `upsert()`.
3. `upsert()` validates the form, processes POST, calls `modelBanner->add($data)`.
4. `ModelContentBanner::add()` inserts the `banner` row (name, `jquery__plugin`, `params`, `publish dates`, `status`).
5. The `on('save')` hook fires: iterates `$data['items']`, calls `setItem()` per slide, which inserts `banner_item` rows + `description` rows (localised titles) + `property` rows (per-item settings).
6. The `object_to_store` rows are written (via the `relations = ["stores"]` declaration) to scope the banner to stores.
7. Admin activity logged.

7.2 Storefront Renders the Banner Widget

1. A page loads with a “banner” widget configured with `banner_id = 5`.
2. `ControllerModuleBanner::init()` registers the `module:settings` filter.
3. `ControllerModuleModuleController::index()` calls the filter.
4. The filter:
 - (a) Calls `modelBanner->getById(5)` — `SELECT * FROM banner b LEFT JOIN object_to_store b2s ON (...) WHERE b.banner_id = 5 AND b.publish_date_start <= NOW() AND ... AND b2s.store_id = STORE_ID.`
 - (b) Calls `modelBanner->getItems(5)` — `SELECT * FROM banner_item WHERE banner_id = 5 AND status = '1'`. Returns the slides.
 - (c) For each slide, loads descriptions (`getDescriptions(banner_item_id)`) and per-item widgets (`NecoWidget::getWidgets(...)` with `object_type = 'banner_item'`).
 - (d) Reads `banner['jquery_plugin']` (e.g. `'nivo-slider'`).
 - (e) Enqueues `sliders/nivo-slider/slider.js` and `sliders/nivo-slider/slider.css`.
 - (f) Sets template to `choroni/banner/nivo-slider.tpl`.
5. `ControllerModuleModuleController::index()` calls `render()`.
6. `fetch('choroni/banner/nivo-slider.tpl')`:
 - (a) Injects magic services (`$Config`, `$Language`, `$l`, `$Request`, `$Url`, `$Image`).
 - (b) Extracts `$this->data` into local scope (`$banner`, `$widgetName`, `$settings`, etc.).
 - (c) Requires the template. The template:
 - Iterates `$banner['items']`, emits `<img>` tags with localised titles.
 - Pushes a config object into `window.ntPlugins`.
 - Calls `loadNTPlugins()`.
 - (d) Substitutes `{%<per-item-widget-name>%}` placeholders with the per-item widget HTML.
 - (e) Returns the HTML.
7. The HTML is returned via the `module:settings` filter and stored in `$this->data['<widget_name>_code']`.
8. The parent page's `fetch()` substitutes the `{%<widget_name>%}` placeholder with the banner widget's HTML.
9. On the client side, `theme.js` runs `loadNTPlugins()` on DOM ready, which applies the NivoSlider plugin to the slider container.

8 Cross-Cutting Findings

8.1 The Banner Subsystem is a Plugin-Aware Widget

The banner subsystem is the only widget module that dynamically selects its template, JS, and CSS based on a database column (`jquery_plugin`). All other widget modules have a fixed template (selected by the admin's `settings['view']` field). The banner widget reads the `jquery_plugin` column from the `banner` table and uses it to look up `choroni/banner/<plugin>.tpl`, `sliders/<plugin>/slider.js`, and `sliders/<plugin>/slider.css`.

8.2 The ntPlugins Registry is the JS Initialisation Seam

The `ntPlugins` registry is the JavaScript initialisation seam for all slider templates. Each template pushes a `{id, config, plugin}` entry, and `loadNTPlugins()` applies the jQuery plugin to the DOM element. This is a decoupled, deferred-initialisation pattern: templates don't need to know whether the jQuery plugin is loaded yet (the `loadNTPlugins()` function checks `$(item.plugin) || $.fn[item.plugin]` before applying).

8.3 Per-Item Widget Composition is Unique to Banners

No other widget module supports per-item widget composition. The banner widget's `module:settings` filter runs a recursive `NecoWidget::getWidgets()` call for each slide, loading widgets scoped to `object_type = 'banner_item'` and `object_id = <banner_item_id>`. These widgets are registered as children of the banner controller and their HTML is substituted into the slide template. The `offsetX/offsetY` settings allow the admin to position widgets as overlays on the slide image.

8.4 The params Column is Effectively Unused

The `banner.params` column stores a serialised PHP array of slider configuration, but the storefront templates hardcode the configuration in the `ntPlugins.push()` call. The `params` column may be used by the admin form for persistence (saving the slider configuration), but it is not read by the storefront. Each template has its own hardcoded config, which means changing the slider speed or effect requires editing the template file, not the admin UI.

8.5 The banner_items Form Array Carries Everything

The admin form submits all slide data (images, links, descriptions, properties) in a single `banner_items` POST array. The model's `on('save')` hook iterates this array and calls `setItem()` per slide, which handles insert-or-update, descriptions, and properties. This is a bulk-save pattern: all slides are saved in one form submission, not via individual AJAX calls.

9 Closing Observations

9.1 The Banner Subsystem is the Most Visually Diverse Part

With 33 templates backed by 14+ jQuery plugins, the banner subsystem is the most visually diverse part of Necoyoad. The `jquery_plugin` discriminator allows the admin to choose a different slider or gallery for each banner, and the storefront dynamically loads the matching template, JS, and CSS. This is a plugin-aware widget architecture that rivals dedicated slider libraries.

9.2 The Per-Item Widget Composition is a Unique Feature

The ability to attach widgets to individual slides — not just to the banner as a whole — is a unique feature. It allows the admin to create rich, interactive slideshows where each slide has overlaid content (product overviews, text blocks, contact forms) positioned at specific coordinates. This is the same widget composition mechanism (v3, v8) applied at the slide level, using the same `NecoWidget::getWidgets()` query with `object_type = 'banner_item'`.

9.3 Final Note

v9 confirms that the Necoyoad banner subsystem is a plugin-aware, widget-composed slideshow system. The `jquery_plugin` discriminator, the 33-template library, the `ntPlugins` registry, and

the per-item widget composition together form a flexible visual content system that can render everything from a simple NivoSlider to a complex parallax gallery with overlaid widgets.

A Glossary

Table 2: Glossary of banner subsystem terms.

Term	Meaning
Banner	A named collection of slides, with a <code>jquery_plugin</code> discriminator.
Banner item	A single slide: image, link, sort_order, status, localised descriptions, per-item properties.
<code>jquery_plugin</code>	The discriminator column that names the jQuery slider plugin to use. Drives template, JS, and CSS selection.
<code>ntPlugins</code>	A JavaScript array registry. Each slider template pushes a <code>{id, config, plugin}</code> entry. <code>loadNTPlugins()</code> applies the plugin on DOM ready.
Per-item widget	A widget attached to an individual slide via <code>object_type = 'banner_item'</code> . Positioned with <code>offsetX/offsetY</code> .
<code>carousel()</code>	An AJAX endpoint that returns banner items as JSON with thumbnails.